MECANIQA API

Guia pratico para explicar como o codigo funciona

Um roteiro de apresentacao para entender a arquitetura, demonstrar os endpoints e responder perguntas tecnicas com seguranca.

Spring Boot | Java | REST | Persistencia em memoria

1. Mensagem central

Comece explicando o problema e o objetivo antes de abrir os arquivos. A audiencia precisa saber o que a API resolve.

Fala sugerida

> *Esta e a API fundacional da MecaniQA. Ela permite cadastrar e gerenciar pecas de estoque e servicos de uma oficina mecanica por meio de endpoints REST. Nesta primeira versao, os dados ficam em memoria, entao o foco esta na organizacao da API e nas regras basicas do negocio.*

Resumo do projeto

- Aplicacao Java com Spring Boot.
- Dados trafegam em JSON por requisicoes HTTP.
- Recursos principais: **pecas** e **servicos**.
- Operacoes disponiveis: criar, consultar, atualizar e remover.

2. Arquitetura em uma frase

```
Cliente HTTP -> Controller REST -> Repository Singleton -> ArrayList em memoria
```

O controller recebe a requisicao, valida o DTO, transforma os dados em um model e chama o repository. O repository manipula a lista compartilhada e devolve o resultado; por fim, o controller monta o status e o JSON da resposta.

Ponto de atencao

> *O projeto nao possui camada **Service**. Isso e intencional para manter o escopo simples: validacao fica no controller e persistencia fica no repository.*

3. Como a aplicacao inicia

Arquivo para mostrar: **ApiApplication.java**

A anotacao **@SpringBootApplication** combina configuracao, descoberta dos componentes e inicializacao da aplicacao. O metodo **main** chama **SpringApplication.run**, que sobe o servidor embutido e disponibiliza a API.

Pergunta: por que nao ha configuracao manual do servidor?

Resposta: o Spring Boot usa convencoes e servidor embutido. Por isso, executar a classe principal ou o Gradle ja e suficiente para iniciar a aplicacao.

4. Modelos de dominio e DTOs

Os models representam o que o sistema armazena. Os DTOs representam o que o cliente pode enviar.

Classe	Responsabilidade
Peca	Representa um item de estoque: identificacao, fornecedor, quantidade, precos, categoria e atributos opcionais.
Servico	Representa um servico: nome, duracao estimada, custo e datas de controle.
CategoriaPeca	Enumera categorias permitidas, como MOTOR, FREIOS e ELETRICA.
PecaRequest / ServicoRequest	Recebem os campos de entrada sem permitir que o cliente defina codigo e datas.

Perguntas frequentes

Por que usar **BigDecimal** nos precos? Para representar dinheiro com precisao, evitando problemas comuns de arredondamento de tipos como **double**.

Por que nao receber o model diretamente? Separar DTO e model impede que o cliente tente controlar campos internos e permite evoluir o contrato de entrada com mais seguranca.

5. Controllers e contrato HTTP

Recurso	Rotas	Status principais
Pecas	POST/GET /api/pecas GET/PUT/DELETE /api/pecas/{codigo}	201, 200, 204, 400, 404
Servicos	POST/GET /api/servicos GET/PUT/DELETE /api/servicos/{codigo}	201, 200, 204, 400, 404

Anotacoes para explicar as anotacoes Spring

- **@RestController**: a classe atende requisicoes e devolve dados, normalmente em JSON.
- **@RequestMapping**: define o prefixo da rota.
- **@PostMapping, @GetMapping, @PutMapping, @DeleteMapping**: associam verbos HTTP a metodos Java.
- **@PathVariable**: captura o codigo da URL.
- **@RequestBody**: converte o JSON recebido em DTO.
- **ResponseEntity**: permite controlar status e corpo da resposta.

6. Validacao e tratamento de respostas

Os controllers validam o corpo antes de chamar o repository. Dados invalidos retornam **400 Bad Request** e nao alteram a lista.

Pecas

O corpo e obrigatorio; codigo de barras e fornecedor ou marca nao podem ficar vazios; estoque e precos nao podem ser negativos; categoria deve ser informada.

Servicos

O corpo e obrigatorio; nome nao pode ficar vazio; duracao deve ser maior que zero; custo nao pode ser negativo.

Fala sugerida

> *Cada status comunica uma situacao diferente: **201** confirma criacao, **200** confirma consulta ou atualizacao, **204** confirma remocao sem corpo e **404** informa que o codigo nao existe.*

7. Repository Singleton

Cada repository tem construtor privado, uma instancia estaticamente criada e o metodo **getInstance()**. Isso faz com que os controllers compartilhem a mesma lista durante a execucao.

Metodo	O que explicar
create	Cria uma copia, gera codigo com AtomicLong, define datas e armazena na lista.
findAll	Retorna uma lista de copias dos registros.
findByCodigo	Procura o codigo e usa Optional para representar encontrado ou ausente.
update	Localiza o item, atualiza campos permitidos e registra a nova data.
delete	Remove pelo codigo e informa se alguma linha foi removida.

Por que **synchronized**? A lista e compartilhada. A sincronizacao evita alteracoes concorrentes inconsistentes, enquanto **AtomicLong** protege a geracao dos codigos.

Por que retornar copias? Para impedir que quem recebeu um objeto altere diretamente o estado interno do repository.

Limitacao que deve ser mencionada

> *Os dados desaparecem quando a aplicacao reinicia. Em uma evolucao, o repository pode ser substituido por uma implementacao com banco de dados sem mudar o objetivo das rotas.*

8. Demonstracao ao vivo

Ordem recomendada: criar, listar, buscar, atualizar, provocar um erro e remover.

Criar uma peca

```
POST http://localhost:8080/api/pecas Content-Type: application/json { "codigoBarras": "7890000000001", "fornecedorMarca": "Bosch", "quantidadeEstoque": 8, "precoCusto": 120.50, "precoVenda": 189.90, "tamanho": "Unico", "cor": "Preto", "categoria": "ELETRICA" }
```

Mostre que o servidor gera **codigo** e datas. Depois use GET para confirmar que o registro esta na lista.

Demonstrar erro

Envie uma quantidade negativa ou retire a categoria. A resposta esperada e 400 com um objeto simples contendo o campo **erro**.

Criar um servico

```
POST http://localhost:8080/api/servicos Content-Type: application/json { "nomeServico": "Troca de oleo", "tempoEstimadoDuracaoMinutos": 40, "custoTabelado": 150.00 }
```

Finalize com DELETE e explique que 204 significa sucesso sem corpo de resposta.

9. Perguntas finais

O que e REST? Um estilo baseado em recursos, URLs e verbos HTTP. Aqui, pecas e servicos sao recursos.

O que e CRUD? Create, Read, Update e Delete: criar, consultar, atualizar e excluir.

Por que Optional? Para deixar explicito que uma busca pode encontrar um registro ou nao encontrar nada.

Por que nao usar @Autowired? O escopo definiu o uso explicito do Singleton via getInstance().

O que falta para producao? Banco de dados, autenticacao, paginacao, tratamento global de erros e testes de endpoints.

Fechamento sugerido

A API entrega um CRUD funcional para pecas e servicos, com separacao entre entrada, dominio, controle HTTP e armazenamento. A implementacao atende ao escopo inicial e deixa claro o caminho para evolucoes futuras.

Para executar os testes: **./gradlew test** ou, no Windows, **.\gradlew.bat test**.